

Harshitha Mogili

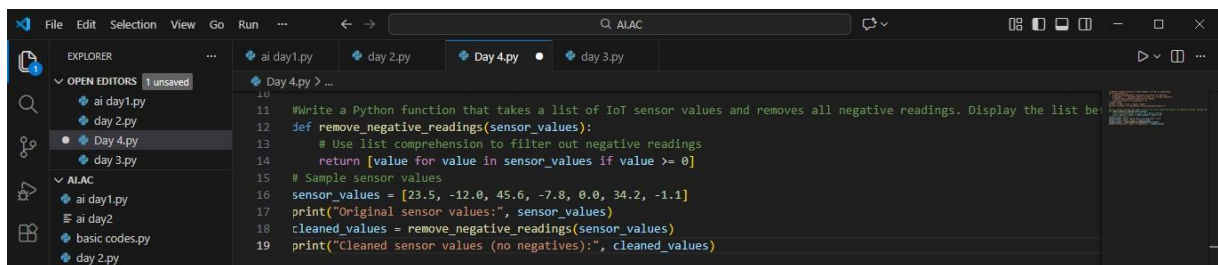
2403A51L01

B-51

ASSIGNMENT -2.2

Task 1: Cleaning Sensor Data

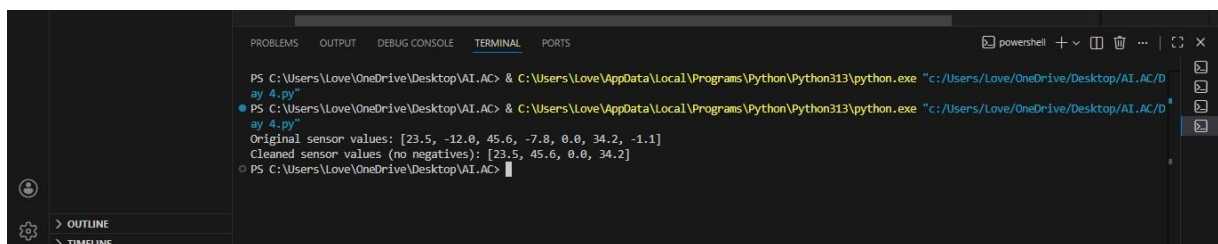
PROMPT: Write a Python function that takes a list of IoT sensor values and removes all negative readings. Display the list before and after cleaning.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project named 'AIAC' with files 'ai day1.py', 'day 2.py', 'Day 4.py', and 'day 3.py'. The code editor shows the following Python code:

```
11 #Write a Python function that takes a list of IoT sensor values and removes all negative readings. Display the list before and after cleaning.
12 def remove_negative_readings(sensor_values):
13     # Use list comprehension to filter out negative readings
14     return [value for value in sensor_values if value >= 0]
15 # Sample sensor values
16 sensor_values = [23.5, -12.0, 45.6, -7.8, 0.0, 34.2, -1.1]
17 print("Original sensor values:", sensor_values)
18 cleaned_values = remove_negative_readings(sensor_values)
19 print("Cleaned sensor values (no negatives):", cleaned_values)
```

OUTPUT:



The screenshot shows a terminal window with the following output:

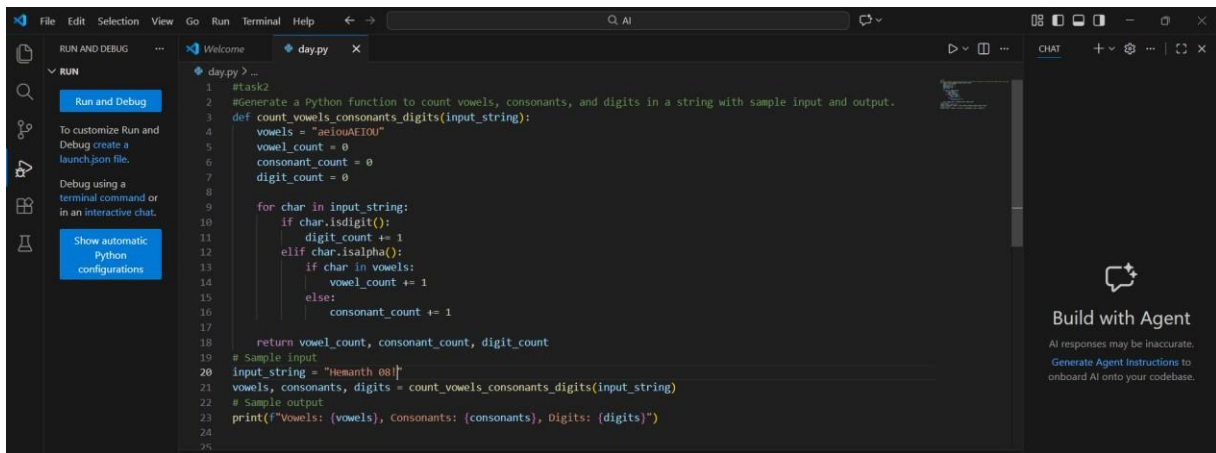
```
PS C:\Users\Love\OneDrive\Desktop\AI.AC> & C:\Users\Love\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/Love/OneDrive/Desktop/AI.AC/Day 4.py"
Original sensor values: [23.5, -12.0, 45.6, -7.8, 0.0, 34.2, -1.1]
Cleaned sensor values (no negatives): [23.5, 45.6, 0.0, 34.2]
```

EXPLANATION:

This function removes invalid negative sensor values using list comprehension. Only values greater than or equal to zero are retained, ensuring clean IoT sensor data.

Task 2: String Character Analysis

PROMPT: Generate a python function to count vowels, consonants and digits in a string with sample input and output.



The screenshot shows a VS Code editor with a Python file named `day.py`. The code defines a function `count_vowels_consonants_digits` that takes an input string and returns a tuple of counts for vowels, consonants, and digits. The function uses `isalpha()` to identify vowels and `isdigit()` to identify digits. A sample input string `"Hemanth 08!"` is used, resulting in 2 vowels, 5 consonants, and 2 digits. The output is printed as `Vowels: (vowels), Consonants: (consonants), Digits: (digits)"`.

```
1 #task2
2 #Generate a Python function to count vowels, consonants, and digits in a string with sample input and output.
3 def count_vowels_consonants_digits(input_string):
4     vowels = "aeiouAEIOU"
5     vowel_count = 0
6     consonant_count = 0
7     digit_count = 0
8
9     for char in input_string:
10         if char.isdigit():
11             digit_count += 1
12         elif char.isalpha():
13             if char in vowels:
14                 vowel_count += 1
15             else:
16                 consonant_count += 1
17
18     return vowel_count, consonant_count, digit_count
19
20 # Sample input
21 input_string = "Hemanth 08!"
22 vowels, consonants, digits = count_vowels_consonants_digits(input_string)
23 # Sample output
24 print(f"Vowels: {vowels}, Consonants: {consonants}, Digits: {digits}")
25
```

OUTPUT:



The screenshot shows a terminal window with the following output:

```
PS C:\Users\girug\Downloads\AI> & 'c:\Users\girug\AppData\Local\Python\pythoncore-3.14-64\python.exe' 'c:\Users\girug\.vscode\extensions\ms-python.debugpy-2025.18.0-x64\libs\debugpy\launcher' '52653' '-.' 'c:\Users\girug\Downloads\AI\day.py'
Vowels: 2, Consonants: 5, Digits: 2
PS C:\Users\girug\Downloads\AI>
```

EXPLANATION:

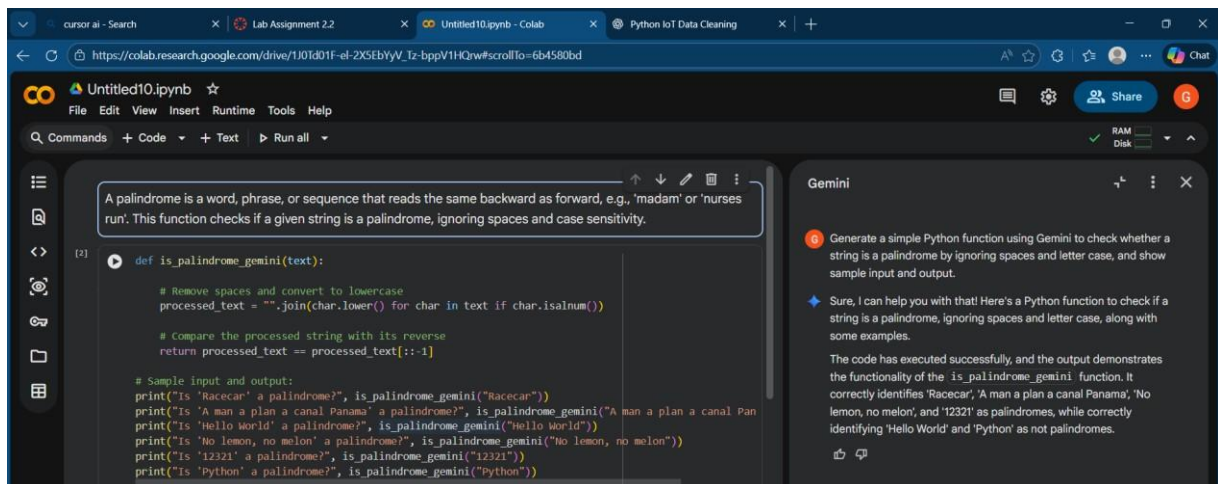
The function iterates through each character and classifies it as a vowel, consonant, or digit.

Python string methods like `isalpha()` and `isdigit()` improve accuracy and readability.

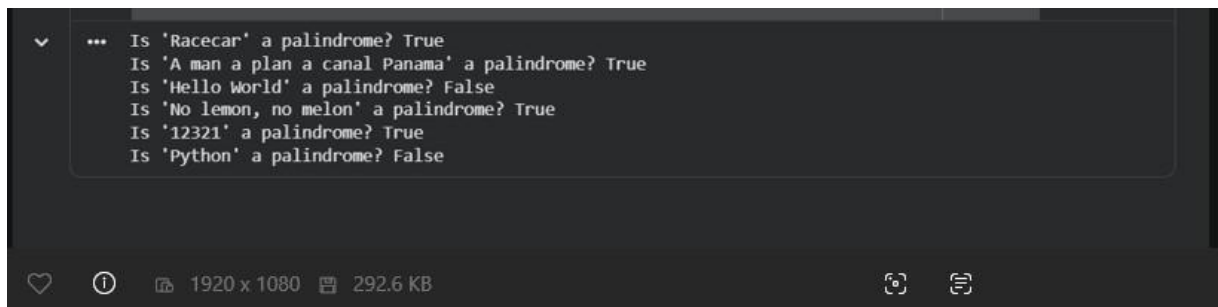
Task 3: Palindrome Check – Tool Comparison

Gemini Prompt: Write a Python function to check if a string is a palindrome.

Ignore spaces and capitalization.



OUTPUT:



Copilot Prompt: Generate a python function to check whether a string is a palindrome (ignore case and spaces) and provide sample input and output.



OUTPUT:

A screenshot of a Python debugger console window. The terminal shows a command prompt where a script is executed. The script checks if 'A man a plan a canal Panama' is a palindrome (True) and if 'Your m!@world' is a palindrome (False). The interface includes tabs for PROBLEMS, OUTPUT, DEBUG CONSOLE, TERMINAL, and PORTS. On the left, there are BREAKPOINTS and a settings icon. On the right, there's a Python Debug Console and a sidebar with a 'dapp' icon and a 'Describe what to build next' button.

Comparison Table:

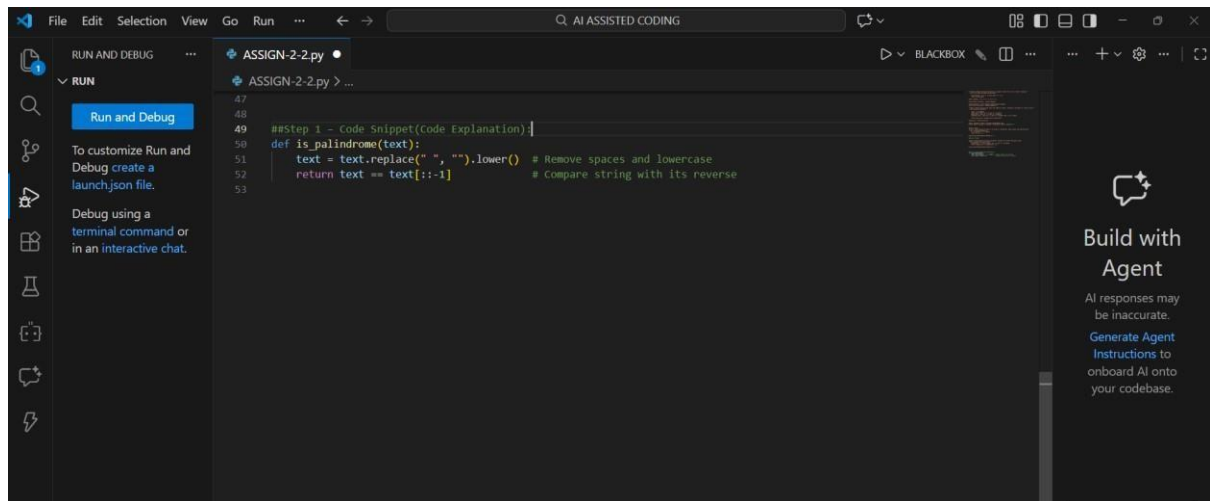
Feature	Gemini	Copilot
Clarity	Simple, minimal code	Slightly longer, more robust
Handling spaces/case	Ignores spaces, converts to lowercase	Ignores spaces and punctuation, lowercase
Readability	Very clear	Clear, slightly more detailed
Efficiency	Uses string slicing	Uses string comprehension

EXPLANATION:

Gemini provides concise and easy-to-read logic, making it beginnerfriendly. Copilot generates more robust code that handles punctuation and special characters.

Task 4: Code Explanation Using AI Step 1 –

Code Snippet:



Step 2 – AI Explanation:

1. `text.replace(" ", "").lower()` → Removes spaces and converts letters to lowercase.
2. `text == text[::-1]` → Checks if the string is equal to its reverse.

EXPLANATION:

The function normalizes the string to avoid case and space mismatches. It then compares the string with its reverse to verify palindrome logic.